

LAPORAN PRAKTIKUM STRUKTUR DATA  
PEKAN 9



Oleh: Afif Naufal Zahran

NIM: 2511533009

DOSEN PENGAMPU: DR.WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI  
DEPARTEMEN INFORMATIKA  
UNIVERSITAS ANDALAS

## **Kata Pengantar**

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah *Struktur Data*. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

## DAFTAR ISI

|                         |  |
|-------------------------|--|
| BAB I.....              |  |
| 1.1 Latar Belakang..... |  |
| 1.2 Tujuan .....        |  |
| 1.3 Manfaat .....       |  |
| BAB II.....             |  |
| 2,1 Teori.....          |  |
| 2,2 Program.....        |  |
| BAB III .....           |  |
| 3,1 Kesimpulan .....    |  |
| 3,2 Saran .....         |  |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Binary tree merupakan struktur data hierarkis di mana setiap titik (node) memiliki maksimal dua cabang anak. Graph adalah struktur data jaringan yang lebih fleksibel, di mana titik-titik saling terhubung tanpa aturan hierarki.

### **1.2 Tujuan**

Tujuan dari laporan praktikum ini agar pembaca dan penulis dapat mendalami dan memahami tentang Binary Tree serta pembuatan Binary Tree java.

### **1.3 Manfaat**

Manfaat dari praktikum tentang operator agar pembaca dan penulis dapat mendalami dan memahami tentang Binary Tree serta pembuatan Binary Tree dan graph di java.

## BAB II

### PEMBAHASAN

#### 2,1 Teori

Binary tree merupakan struktur data hierarkis di mana setiap titik (node) memiliki maksimal dua cabang anak. Graph adalah struktur data jaringan yang lebih fleksibel, di mana titik-titik saling terhubung tanpa aturan hierarki.

#### 2,2 Program

Pada pekan ke-9 penulis telah membuat beberapa program yakni Node\_2511533009.java, BTree\_2511533009, BtreeDriver\_2511533009, dan GraphTraversal\_2511533009.

##### a. Node\_2511533009.java

```
package pekan9_2511533009;

public class Node_2511533009 {
    int data_3009;
    Node_2511533009 left_3009;
    Node_2511533009 right_3009;
    public Node_2511533009(int data_3009){
        this.data_3009=data_3009;
        left_3009 = null;
        right_3009 = null;
    }

    public void setLeft_3009(Node_2511533009 node_3009){
        if (left_3009 == null)
            left_3009 = node_3009;
    }
    public void setRight_3009(Node_2511533009 node_3009){
        if (right_3009 == null)
            right_3009 = node_3009;
    }
    public Node_2511533009 getLeft_3009(){
        return this.left_3009;
    }
    public Node_2511533009 getRight_3009(){
        return this.right_3009;
    }
    public int getData_3009(){
```

```

        return data_3009;
    }
    public void setData_3009(int data_3009){
        this.data_3009 = data_3009;
    }
    void printPreorder_3009(Node_2511533009 node_3009){
        if (node_3009 == null)
            return;
        System.out.printf(node_3009.data_3009 + " ");
        printPreorder_3009(node_3009.left_3009);
        printPreorder_3009(node_3009.right_3009);
    }

    void printPostorder_3009(Node_2511533009 node_3009){
        if (node_3009 == null )
            return;
        printPostorder_3009(node_3009.left_3009);
        printPostorder_3009(node_3009.right_3009);
        System.out.printf(node_3009.data_3009 + " ");
    }

    void printInorder_3009(Node_2511533009 node_3009){
        if (node_3009 == null)
            return;
        printInorder_3009(node_3009.left_3009);
        System.out.printf(node_3009.data_3009 + " ");
        printInorder_3009(node_3009.right_3009);
    }

    public String print_3009(){
        return this.print_3009("",true,"");
    }

    public String print_3009(String prefix_3009, boolean
isTail_3009,String sb_3009){
        if (right_3009 != null){
            right_3009.print_3009(prefix_3009 + (isTail_3009 ? "|"
: " : " ),false,sb_3009);
        }
        System.out.println(prefix_3009 + (isTail_3009 ? "\\-- "
: "/-- ") + data_3009);
        if (left_3009 != null){
            left_3009.print_3009(prefix_3009 + (isTail_3009 ? " "
: "| "),true,sb_3009);
        }
        return sb_3009;
    }

```

```
}
```

Kode di atas merupakan class object Node\_2511533009 dimana class ini memiliki 3 attribute yakni data dengan tipe data integer, left\_3009 dengan tipe data Node\_2511533009 dan right\_3009 dengan tipe data yang sama dengan left\_3009 berguna untuk pointer ke cabang kanan atau cabang kiri dari pohon binary.

b. BTree\_2511533009.java

```
package pekan9_2511533009;

import org.w3c.dom.Node;

public class BTree_2511533009 {
    private Node_2511533009 root_3009;
    private Node_2511533009 currentNode_3009;
    public BTree_2511533009(){
        root_3009 = null;
    }
    public boolean search_3009(int data_3009){
        return search_3009(root_3009,data_3009);
    }
    private boolean search_3009(Node_2511533009 node_3009,int
data_3009){
        if (node_3009.getData_3009() == data_3009)
            return true;
        if (node_3009.getLeft_3009() != null)
            if (search_3009(node_3009.getLeft_3009(),data_3009))
                return true;
        if (node_3009.getRight_3009() != null)
            if (search_3009(node_3009.getRight_3009(),data_3009))
                return true;
        return false;
    }
    public void printInorder_3009(){
        root_3009.printInorder_3009(root_3009);
    }
    public void printPreOrder_3009(){
        root_3009.printPreorder_3009(root_3009);
    }
    public void printPostOrder_3009(){
        root_3009.printPostorder_3009(root_3009);
    }

    public Node_2511533009 getRoot_3009(){
```

```

        return root_3009;
    }

    public boolean isEmpty_3009(){
        return root_3009 == null;
    }

    public int countNodes_3009(){
        return countNodes_3009(root_3009);
    }

    public int countNodes_3009(Node_2511533009 node_3009){
        int count_3009 = 1;
        if (node_3009 == null){
            return 0;
        } else {
            count_3009 += countNodes_3009(node_3009.getLeft_3009());
            count_3009 += countNodes_3009(node_3009.getRight_3009());
            return count_3009;
        }
    }

    public void print_3009(){
        root_3009.print_3009();
    }

    public Node_2511533009 getCurrentNode_3009(){
        return currentNode_3009;
    }

    public void setCurrentNode_3009(Node_2511533009 currentNode_3009)
{
        this.currentNode_3009 = currentNode_3009;
    }

    public void setRoot_3009(Node_2511533009 root_3009) {
        this.root_3009 = root_3009;
    }
}

```

Kode di atas merupakan class object BTree yang memiliki atribut root\_3009 dengan tipe data class object Node\_251153309, dan currentNode\_3009. Pada object tersebut terdapat beberapa fungsi, namun pada kali ini kita akan fokus terhadap fungsi search\_3009 yang memiliki 2 parameter yakni node\_3009 dan data\_3009. Fungsi ini akan melakukan pengecekan jika data dari node ada, jika ada akan mengembalikan



true, jika tidak dan data node kiri tidak kosong dan search dengan parameter mengembalikan nilai true, maka fungsi akan mengembalikan nilai true, jika tidak melakukan pengecekan jika cabang kanan tidak kosong dan nilai dari fungsi search dengan 2 parameter mengembalikan nilai true maka fungsi akan mengembalikan nilai true.

c. BtreeDriver\_2511533009

```
package pekan9_2511533009;

public class BtreeDriver_2511533009 {
    public static void main(String[] args){
        //Membuat Pohon
        BTree_2511533009 tree_3009 = new BTree_2511533009();
        System.out.printf("umlah Simpul awal pohon: ");
        System.out.println(tree_3009.countNodes_3009());
        //menambahkan simpul data 1
        Node_2511533009 root_3009 = new Node_2511533009(1);
        tree_3009.setRoot_3009(root_3009);
        //menjadikan simpul 1 sebagai root
        System.out.println("Jumlah simpul jika hanya ada root");

        System.out.println(tree_3009.countNodes_3009());
        Node_2511533009 node2_3009 = new Node_2511533009(2);
        Node_2511533009 node3_3009 = new Node_2511533009(3);
        Node_2511533009 node4_3009 = new Node_2511533009(4);
        Node_2511533009 node5_3009 = new Node_2511533009(5);
        Node_2511533009 node6_3009 = new Node_2511533009(6);
        Node_2511533009 node7_3009 = new Node_2511533009(7);
        Node_2511533009 node8_3009 = new Node_2511533009(8);
        Node_2511533009 node9_3009 = new Node_2511533009(9);
        root_3009.setLeft_3009(node2_3009);
        node2_3009.setLeft_3009(node4_3009);
        node2_3009.setRight_3009(node5_3009);
        node4_3009.setRight_3009(node8_3009);
        root_3009.setRight_3009(node3_3009);
        node3_3009.setLeft_3009(node6_3009);
        node3_3009.setRight_3009(node7_3009);
        node6_3009.setLeft_3009(node9_3009);
        //Set root
        tree_3009.setCurrent_3009(tree_3009.getRoot_3009());
        System.out.println("Menampilkan simpul treakihr: ");

        System.out.println(tree_3009.getCurrentNode_3009().getData_3009());
```

```

        System.out.println("Jumlah simpul; setelah simmpul 7
ditambahkan");
        System.out.println(tree_3009.countNodes_3009());
        System.out.println("InOrder: ");
        tree_3009.printInorder_3009();
        System.out.println("\nPreorder: ");
        tree_3009.printPreOrder_3009();
        System.out.println("\nPostorder:");
        tree_3009.printPostOrder_3009();
        System.out.println("\nDMenampilkan simpul dalam bentuk
pohon");
        tree_3009.print_3009();
    }
}

```

Kode di atas merupakan kode driver dari class object BTree\_2511533009 dimana penulis membuat tree\_3009 dengan root root\_3009 dan 9 nde sebagai cabang kanan ataupun kiri.

d. GraphTraversal\_2511533009

```

package pekan9_2511533009;

import java.util.*;

public class GraphTraversal_2511533009 {
    private Map<String, List<String>> grap_3009 = new HashMap<>();

    //Menambakan edge (graf tak berarah)
    public void addEdge_3009(String node1_3009, String node2_3009){
        grap_3009.putIfAbsent(node1_3009, new ArrayList<>());
        grap_3009.putIfAbsent(node2_3009, new ArrayList<>());
        grap_3009.get(node1_3009).add(node2_3009);
        grap_3009.get(node2_3009).add(node1_3009);
    }

    /// Menampilkan graf awal
    public void printGraph_3009(){
        System.out.println("Graf Awal (Adjacency List): ");
        for (String node_3009 : grap_3009.keySet()){
            System.out.printf(node_3009 + " -> ");
            List<String> neighbors_3009 = grap_3009.get(node_3009);
            System.out.println(String.join(", ", neighbors_3009));
        }
        System.out.println();
    }
}

```

```

//DFS rkursif
public void dfs_3009(String start_3009){
    Set<String> visited_3009 = new HashSet<>();
    System.out.println("Penelusuran DFS: ");
    dfsHelper_3009(start_3009,visited_3009);
    System.out.println();
}
private void dfsHelper_3009(String current_3009, Set<String>
visited_3009){
    if (visited_3009.contains(current_3009)) return;
    visited_3009.add(current_3009);
    System.out.printf(current_3009 + " ");
    for (String neiighbor_3009 :
grap_3009.getDefault(current_3009,new ArrayList<>())){
        dfsHelper_3009(neiighbor_3009,visited_3009);
    }
}

//BFS iteratif
public void bfs_3009(String start_3009){
    Set<String> visited_3009 = new HashSet<>();
    Queue<String> queue_3009 = new LinkedList<>();
    queue_3009.add(start_3009);
    visited_3009.add(start_3009);
    System.out.println("Penelusuran BFS: ");
    while (!queue_3009.isEmpty()){
        String current_3009 = queue_3009.poll();
        System.out.print(current_3009 + " ");
        for (String neighbor_3009 :
grap_3009.getDefault(current_3009,new ArrayList<>())){
            if(!visited_3009.contains(neighbor_3009)) {
                queue_3009.add(neighbor_3009);
                visited_3009.add(neighbor_3009);
            }
        }
    }
    System.out.println();
}

//Main
public static void main(String [] args){
    GraphTraversal_2511533009 graph_3009 = new
GraphTraversal_2511533009();

    //Cntoh graf: A-B, A-C, B-D, B-E

```

```

graph_3009.addEdge_3009("A","B");
graph_3009.addEdge_3009("A","C");
graph_3009.addEdge_3009("B","D");
graph_3009.addEdge_3009("B","E");
//Cetak graf awal
System.out.println("Garf Awal adalah: ");
graph_3009.printGraph_3009();
// Lakukan penelusuran
graph_3009.bfs_3009("A");
graph_3009.bfs_3009("A");
}
}

```

Kode diatas merupakan kode GraphTraversal dimana terdapat Map<String, ArrayList<String>>> sebagai graph. Lalu terdapat beberapa method seperti addEdge, printGrah, dfsHhelper, bfs dan main untuk menjalankan kode nya, di fungsi main kita memanggil funngsi addEdge\_3009() untuk membuat graf A-B, A-C, B-D, dan B-E, dan melakukan penelusuran bfs dan dfs yang dimulai dari “A”.

## **BAB III**

### **PENUTUPAN**

#### 3,1 Kesimpulan

Binary Tree dan Graph cukup penting pada bahasa pemrograman baik itu *Java* ataupun bahasa pemrograman yang lain karena hal tersebut sangat fundamental pada bahasa pemrograman dan struktur data.

#### 3,2 Saran

Laporan praktikum ini masih memiliki kekurangan. Karena itu, penulis sangat terbuka terhadap saran dan kritikan agar dapat meningkatkan kualitas laporan ini dan laporan-laporan selanjutnya.